ScamLens AI: An Intelligent NLP-Based Scam Detection and Risk ClassificationSystem Using TF-IDF Feature Extraction and Supervised Machine Learning

Saranya V
Reg. No. 240701620
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
saranyavimalanathan.cse.2024@rajalakshmi.edu.in

Gayathri Jayam S
Reg. No. 240701142
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
gayathrijayam.s.cse.2024@rajalakshmi.edu.in

***Abstract*—Organ transplantation depends on the timely coordination of donor-organ availability, recipient compatibility, hospital resources, and time-critical transport, yet these details are often scattered across separate records, which makes candidates and transport options hard to compare and explain. This paper presents IntelliTransplant, a web-based decision-support prototype that unifies organ, recipient, and hospital records with five cooperating components: a knowledge engine of eligibility rules, a Constraint Satisfaction Problem (CSP) engine with backtracking for organ–recipient matching, A* and Uniform Cost Search (UCS) for inter-hospital route planning, a Bayesian engine for transport-risk estimation, and a coordination agent that orchestrates them. Matching returns a compatibility score together with a decision explanation generated from the checks that were evaluated, and transport planning returns a route, distance, estimated travel time, and risk and success estimates. The system uses a React frontend, a FastAPI backend with Pydantic validation, and a SQLAlchemy-managed relational database, with donor identifiers generated on the server. We evaluate the prototype through an end-to-end demonstration on a seeded environment of six organs, four recipients, and four hospitals in Chennai. The matching module selects a kidney–recipient pair with a compatibility score of 99, and the transport module plans a 3.4 km A* route whose distance agrees with the great-circle distance (3.37 km) that we recompute independently from the stored hospital coordinates. We also report the open issues that this demonstration exposes: an undocumented score range, uncalibrated risk estimates, and the absence of live traffic data. IntelliTransplant is an academic demonstrator and not a clinical allocation tool.**

***Index Terms*—organ transplantation, clinical decision support, constraint satisfaction, backtracking search, A* search, uniform cost search, Bayesian reasoning, explainable AI, healthcare logistics, FastAPI, React.**

I. INTRODUCTION

Organ transplantation is among the most coordination-intensive processes in healthcare. A single transplant involves identifying a donor organ, evaluating waiting recipients, assessing compatibility, communicating between hospitals, and moving the organ within a limited time window [1], [2]. The information needed for these activities is frequently held in separate records or exchanged manually, so a coordinator must repeatedly switch between sources to compare candidates, justify a recommendation, and plan movement between hospitals.

Two difficulties follow. First, compatibility assessment mixes hard eligibility rules (for example, the required organ type and blood-group compatibility) with softer ranking criteria (for example, urgency and sensitisation). When both are folded into one opaque number, the reasons behind a recommendation are hard to inspect. Second, transport planning is usually handled separately from matching, so route cost and risk are not visible at the moment a candidate pair is considered.

This paper presents IntelliTransplant, a web-based decision-support prototype that addresses both difficulties with classical, inspectable AI techniques rather than learned models. Organ, recipient, and hospital records are managed through one interface; a CSP engine with backtracking selects and scores organ–recipient pairs; A* search and UCS plan routes between hospitals; a Bayesian engine produces a transport-risk estimate; and a coordination agent chains these components and assembles the result for the user. The prototype is intended for academic demonstration. It is not a clinical allocation authority and does not replace transplant-team judgement or validated medical protocols.

The core contributions of this work are:

(i) A unified, database-backed coordination workflow with organ, recipient, hospital, matching, and transport modules, in which identifiers are generated by the backend and the interface renders only persisted records;

(ii) A CSP formulation of organ–recipient matching that separates hard eligibility constraints from score-based ranking, solved by backtracking, with an explanation built from the checks actually evaluated;

(iii) Hospital-to-hospital route planning with A* search over a graph built from stored coordinates, with UCS as a cost-based cross-check;

(iv) A Bayesian transport-risk estimate that is explicitly presented as a prototype estimate rather than a validated probability; and

(v) An end-to-end demonstration with an independent verification of the reported route distance, together with a candid analysis of the prototype's limitations.

The remainder of this paper is organized as follows: Section II reviews related work; Section III presents the formal problem model; Section IV describes the proposed method; Section V outlines the system implementation; Section VI reports the demonstration results and limitations; and Section VII concludes.

II. RELATED WORK

A. Organ Allocation and Transplant Logistics

Real allocation systems combine medical compatibility, urgency, waiting time, and geography, and publish the factors they use so that the process is open to scrutiny [1]. International guidance stresses ethical, transparent, and accountable practice in transplantation [2]. Market-design research has also formalised transplant matching, notably

kidney exchange, as an optimisation problem with explicit constraints [3]. Our work does not propose an allocation policy. It is an engineering demonstrator of how rule-based matching, route planning, and uncertainty estimation can sit in one coordination interface.

B. Constraint Satisfaction Problems

A CSP is defined by variables, domains, and constraints [4], [5]. Candidate pairs can first be filtered by hard constraints such as required organ type and blood-group compatibility, and the feasible set can then be ranked. Backtracking search with constraint propagation is the standard way to solve such problems [6], [7]. CSPs are attractive here because eligibility conditions are explicit rather than hidden inside a score.

C. Heuristic Search and Path Planning

UCS expands the frontier node with the lowest accumulated cost and is closely related to Dijkstra's shortest-path algorithm [8]. A* adds a heuristic estimate of the remaining cost and keeps optimality when that heuristic is admissible [9]. For geographic graphs, the great-circle (haversine) distance is a common heuristic and edge cost [10]. Textbooks treat both algorithms as graph-search methods that return the same optimal cost when edge costs and heuristics are well defined [5], [11], [12].

D. Bayesian Reasoning

Bayesian reasoning updates the probability of a hypothesis from prior information and observed evidence [13]. In a prototype, it can turn configured factors into an estimated transport risk. Such a value is meaningful only to the extent that the variables, priors, and conditional probabilities are defined and validated, so it should not be read as a medically validated probability.

E. Explainability and Web Technologies

Research on explanation argues that people want to see the reasons behind a decision, not only the decision [14]. We therefore build explanations from the evaluated checks. For delivery, the stack uses React for state-driven interfaces and FastAPI with Pydantic and SQLAlchemy for validated REST endpoints and persistence [15], [16], [17].

III. PROBLEM FORMULATION

A. System Model and Records

Let H, O, and R be the sets of registered hospitals, organs, and recipients. Each entity is a tuple:

$$h = (id_h, name, city, \varphi, \lambda, c) \quad (1)$$

$$o = (id_o, \tau_o, \beta_o, \delta_o, h_o, s_o, t_o) \quad (2)$$

$$r = (id_r, n_r, a_r, \tau_r, \beta_r, u_r, \pi_r, h_r) \quad (3)$$

where φ and λ are the latitude and longitude of a hospital and c its contact number. For an organ, τ is the organ type, β the blood group, δ the donor identifier generated by the backend, h_o the hospital association, s the status (for example, available), and t the creation time. For a recipient, n is the name, a the age, τ_r the required organ, $u_r \in \{1,\ldots,5\}$ the urgency level, $\pi_r \in [0,100]$ the PRA percentage, and h_r the registered hospital.

B. Objectives and Constraints

The prototype must satisfy four requirements: (1) traceability, meaning every recommendation is shown with the checks and values that produced it; (2) data integrity, meaning identifiers are generated by the backend, the database is the source of truth, and the frontend renders returned records instead of locally created rows; (3) timeliness, meaning a transport recommendation is computed from hospital locations and shown with distance and estimated time; and (4) a safety boundary, meaning outputs are labelled as decision support and estimates are labelled as estimates. Formally, the system seeks the highest-scoring feasible organ–recipient pair and the minimum-cost route between two hospitals.

C. Matching Constraints and Score

A pair is feasible when the organ is available, the organ type satisfies the recipient's need, and the blood groups are compatible:

$$\mathrm{Feas}(o,r) = 1[s_o = \text{available}] \cdot 1[\tau_o = \tau_r] \cdot 1[(\beta_o, \beta_r) \in K] \quad (4)$$

where K is the configured blood-group compatibility relation of the knowledge engine. Feasible pairs are ranked by a score,

$$S(o,r) = \mathrm{Feas}(o,r) \cdot \Sigma_k \, w_k \, \psi_k(o,r) \quad (5)$$

where the component functions ψ_k draw on attributes such as the urgency level u_r and the PRA percentage π_r, and w_k are weights. The weights and the maximum score are implementation parameters of the backend and are reported as returned, not assumed. The selected pair is

$$(o^*, r^*) = \arg\max_{(\{i:o,r) \in F\}} S(o,r) \quad (6)$$

with $F = \{(o,r) : \mathrm{Feas}(o,r) = 1\}$.

D. Transport Cost and Risk

Hospitals are nodes of a graph $G = (H, E)$. The distance-based edge cost between hospitals i and j is the great-circle distance,

$$d_{ij} = 2R \arcsin \sqrt{[\, \sin^2(\Delta\varphi/2) + \cos\varphi_i \cos\varphi_j \sin^2(\Delta\lambda/2) \,]} \quad (7)$$

with Earth radius $R = 6371$ km. The cost of a route P and the optimal route are

$$C(P) = \Sigma_{(\{i:i,j) \in P\}} \, c_{ij}, \quad P^* = \arg\min C(P) \quad (8)$$

with $c_{ij} = d_{ij}$ in the distance-based configuration. The engine also outputs a transport-risk estimate $\rho \in [0,1]$ and a success estimate $1 - \rho$. Estimated travel time is derived by the backend from the route and is not live-traffic data.

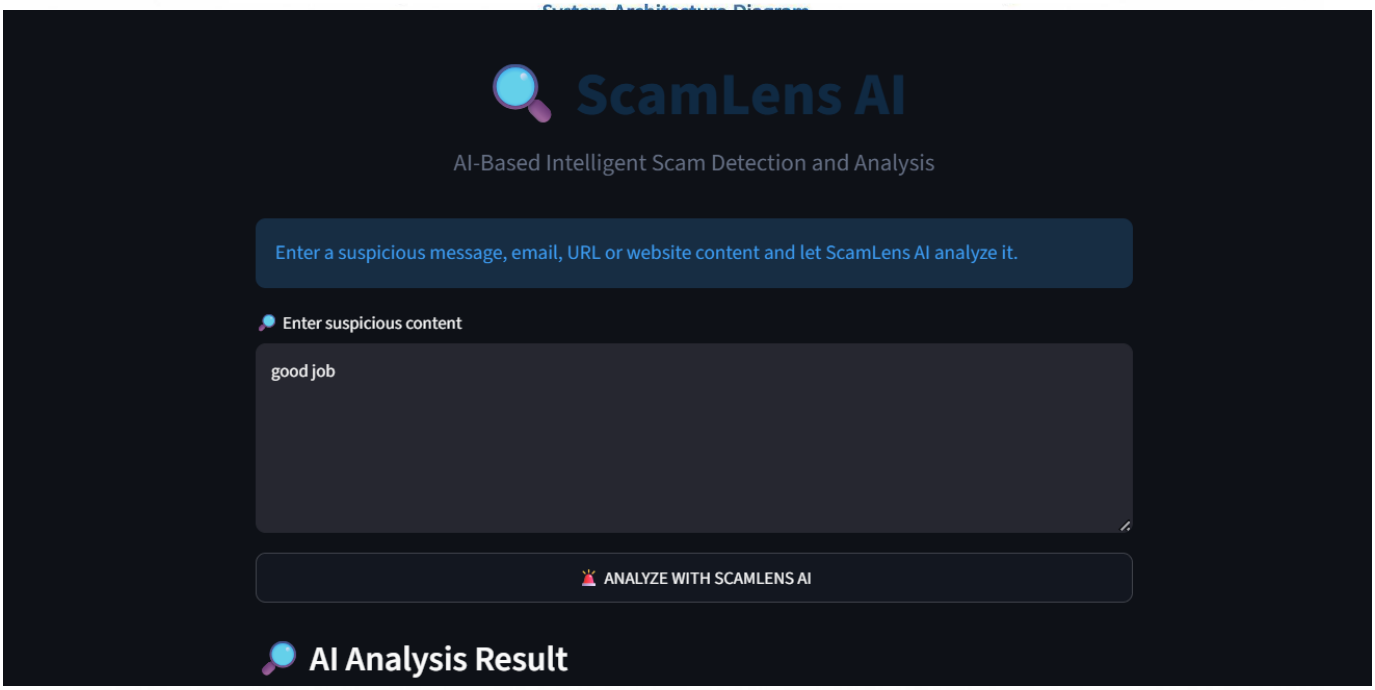


Fig. 1. System architecture of IntelliTransplant showing the user-interface, backend-API, classical-AI decision-engine, and database layers, and the matching and transport workflows joined by the coordination agent.

IV. PROPOSED METHOD

A. Knowledge Engine

The knowledge engine stores the domain rules used by the workflow: organ categories, the blood-group compatibility relation K, and field definitions. It keeps hard eligibility constraints separate from soft ranking preferences, so that the CSP engine can treat the first as filters and the second as score terms. In a deployed system these rules would need to be traceable to authorised medical sources.

B. CSP Engine with Backtracking

The CSP engine treats each available organ as a variable whose domain is the set of recipients, plus "unassigned". The constraints are those of Eq. (4). Backtracking assigns variables one at a time, tests the constraints after each assignment, prunes branches that violate them, and backs up when no value remains [5], [6]. Every constraint outcome is recorded. The engine then ranks the feasible set with Eq. (5) and returns the best pair of Eq. (6). In the general assignment form, a recipient may additionally be limited to one organ per run.

The decision explanation shown to the user is assembled from the recorded outcomes (organ-type compatibility, blood-group compatibility, urgency level, and PRA value) rather than from a fixed sentence, so the explanation cannot disagree with the result it describes.

C. Route Planning with A and UCS*

A* evaluates a node n with

$$f(n) = g(n) + h(n) \tag{9}$$

where $g(n)$ is the cost accumulated from the origin and $h(n)$ estimates the remaining cost to the destination. With the great-circle distance of Eq. (7) as h, the heuristic never exceeds the true remaining cost under distance-based edges, so it is admissible and A* returns an optimal route [9]. UCS is the special case $h \equiv 0$. Because both are optimal for non-negative costs, they must return equal route costs on the same graph. We therefore use UCS as a cross-check: a difference between the two costs signals a heuristic or implementation fault, whereas the routes themselves may differ when several routes tie.

D. Bayesian Risk Estimation

Let $\rho = 1$ denote an adverse transport outcome and $e = (e_1,\ldots,e_k)$ the configured risk factors. By Bayes' rule, and under a conditional-independence (naive Bayes) assumption,

$$P(\rho=1 \mid e) = P(e \mid \rho=1)P(\rho=1) \,/\, P(e) \tag{10}$$

$$P(\rho=1 \mid e) \propto P(\rho=1)\, \Pi_m\, P(e_m \mid \rho=1) \tag{11}$$

The success estimate is the complement $1 - \rho$. The variables, priors, and conditional probabilities are configuration of the engine. Its outputs are therefore labelled model estimates, and they are not claimed to be clinically validated.

E. Coordination Agent

The coordination agent is the workflow orchestrator. It retrieves the relevant records, invokes the matching and transport components in sequence, preserves which component produced each value, and handles missing or failed engine responses explicitly. Algorithm 1 summarises the pipeline, and Table I lists the role of each component. The components have distinct jobs and do not compute the same score five times: the knowledge engine supplies rules, the CSP engine handles matching, search handles route cost, Bayesian reasoning handles uncertainty, and the agent connects them.

Algorithm 1. Match-and-Transport Coordination Pipeline
1: INPUT organs O, recipients R, hospitals H, origin h_s, destination h_d
2: $F \leftarrow \emptyset$
3: for each available organ o in O do
4: for each recipient r in R do
5: if Feas(o,r) = 1 then add (o,r) to F; keep check results
6: if $F = \emptyset$ then return "no feasible match"
7: (o^*,r^*) ← arg max $S(o,r)$ over F; explanation ← kept check results
8: $P_{A^*} \leftarrow$ AStar(G,h_s,h_d); $P_{UCS} \leftarrow$ UCS(G,h_s,h_d)

9: if $C(P_{A^*}) \neq C(P_{UCS})$ then flag inconsistency
10: $\rho \leftarrow$ BayesRisk(evidence(P_{A^*}))
11: persist match and transport records; return result with provenance

TABLE I
ROLES OF THE AI COMPONENTS

Component	Technique	Input → output
Knowledge engine	Rule base	Field and organ data → hard rules, compatibility relation
CSP engine	CSP + backtracking	Organs, recipients → feasible pairs, score, explanation
A*/UCS engine	Graph search	Hospital coordinates → route, distance, time
Bayesian engine	Bayes' rule	Configured factors → risk and success estimates
Coordination agent	Orchestration	Engine outputs → assembled result with provenance

V. SYSTEM IMPLEMENTATION

A. Architecture

The platform is a client–server web application organised in four layers (Fig. 1). The user layer is a React frontend built with Vite, with Dashboard, Organ, Recipient, Hospital, Matching, and Transport pages. The backend layer is a FastAPI REST API with Organ, Recipient, Hospital, Matching, and Transport routes. The decision-engine layer holds the five components of Table I. The database layer is a SQLite store that holds hospitals, organs and donor details, recipients, match results, and transport records; audit logs are a planned module.

B. Data Model and Identifiers

The organs table has an integer primary key, organ type, blood group, donor identifier, optional donor name and contact, hospital identifier, status, and creation time. The donor identifier is not an input field: the backend generates it when the record is created and returns it in the POST response, so the frontend never invents identifiers. The hospital table shows readable codes such as HSP-0001, while the organ schema uses an integer hospital identifier. The displayed code therefore maps to an internal numeric key, and requests must send the type the schema expects.

C. API and Data Flow

A user action in a React page calls a service function that sends an HTTP request [17] to a FastAPI route. Pydantic schemas validate the request fields, and the route uses a SQLAlchemy session to query or commit records [15]. For organs, GET /organs/ returns the stored records and POST /organs/ adds, commits, refreshes, and returns one record as an OrganResponse. The response is serialised to JSON, and the page reloads its list so that the table always reflects the database [16]. When the API fails, the interface shows the failure and does not silently substitute sample data.

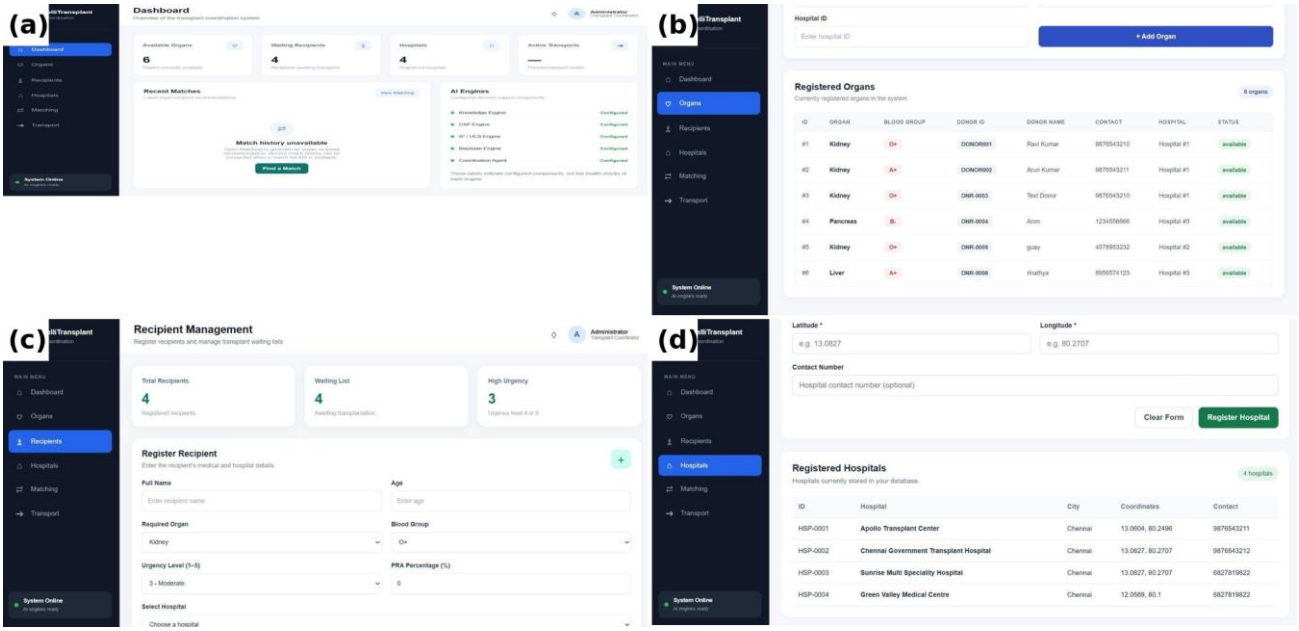


Fig. 2. Application screens: (a) dashboard with summary cards and the list of configured AI engines; (b) organ registration and registered-organ table; (c) recipient registration with waiting-list summary; (d) hospital registration and registered-hospital table.

D. User Interface Modules

Fig. 2 shows the main screens. The dashboard summarises available organs, waiting recipients, registered hospitals, and active transports, and lists the five configured components. The Organ page separates data entry from the registered-organ table. The Recipient page captures name, age, required organ, blood group, urgency level (1–5), PRA percentage, and hospital, and shows total, waiting, and high-urgency counts. The Hospital page stores name, city, latitude, longitude, and an optional contact number. The Matching and Transport pages display the engine outputs described in Section VI. The dashboard labels engines as "configured", which means they are represented in the application and does not by itself prove that each has been independently validated.

E. Validation Approach

The planned checks are: confirm that navigation works on every page; load organ, recipient, and hospital lists and compare them with API responses; submit valid and invalid forms and inspect the status and message; create a record and refresh to confirm persistence; run matching and routing on known demonstration records and inspect the explanations; and verify that failures are displayed instead of misleading placeholders. At the time of writing, the results of this paper come from a single demonstration environment, and no systematic performance, clinical, or user-study data have been collected.

VI. DEMONSTRATION AND RESULTS

A. Experimental Setup

The prototype was run on a seeded demonstration database (Table II). All records are demonstration data, and the hospitals are examples in Chennai that must not be read as verified operational transplant centres. The counts describe the state captured at the time of the screenshots and are not guaranteed for future runs.

TABLE II
DEMONSTRATION ENVIRONMENT

Entity	Count	Remark
Organs	6	4 kidney, 1 pancreas, 1 liver; all available
Recipients	4	4 on waiting list; 3 with urgency 4–5
Hospitals	4	Chennai; coordinates and contacts stored
Transport plans	1 shown	Transport #16, status planned

B. Record Management

The registered-organ table in Fig. 2(b) shows six persisted records with automatically assigned organ and donor identifiers. The donor-identifier format changes from DONOR00*n* for the first two records to DNR-000*n* for records 3 to 6, which suggests that the first two were entered before server-side generation was introduced. A single enforced format with a uniqueness constraint would remove this inconsistency. The recipient and hospital screens show four records each, and their counts match the dashboard cards in Fig. 2(a).

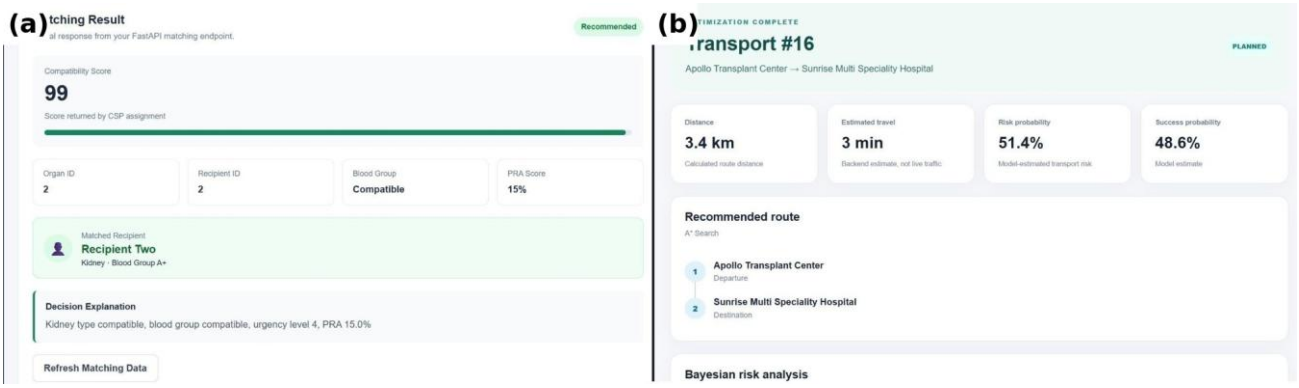


Fig. 3. Engine outputs returned by the backend: (a) matching result with compatibility score 99, organ 2, recipient 2, and decision explanation; (b) Transport #16 planned with A* between Apollo Transplant Center and Sunrise Multi Speciality Hospital, with distance, estimated time, and risk and success estimates.

C. Matching Result

The Matching page requests a match from the backend and displays the outcome in Fig. 3(a) and Table III. The selected organ (ID 2) is a kidney of blood group A+ and the matched recipient (ID 2) requires a kidney and has blood group A+. The returned explanation lists organ-type compatibility, blood-group compatibility, urgency level 4, and a PRA of 15.0%, which agrees with the checks of Eq. (4) and with the stored records. The compatibility score is 99. The interface labels it as the score returned by the CSP assignment, and the maximum possible value and the weights of Eq. (5) are not displayed, so we do not interpret 99 as a percentage.

TABLE III
MATCHING OUTPUT (FIG. 3A)

Field	Value
Compatibility score	99 (CSP assignment)
Organ	ID 2: kidney, A+
Recipient	ID 2: kidney, A+
Blood group status	Compatible
PRA / urgency	15% / level 4
Status	Recommended

D. Transport Result and Independent Verification

Fig. 3(b) and Table IV show Transport #16 between Apollo Transplant Center (HSP-0001) and Sunrise Multi Speciality Hospital (HSP-0003), planned with A* as a two-node route.

TABLE IV
TRANSPORT OUTPUT (FIG. 3B)

Field	Value
Route	HSP-0001 → HSP-0003
Method / status	A* search / planned
Distance	3.4 km
Estimated travel	3 min (not live traffic)
Risk / success	51.4% / 48.6%

To verify the distance independently, we recomputed the great-circle distance of Eq. (7) from the stored coordinates of Fig. 2(d). Table V gives all pairwise values. The HSP-0001 to HSP-0003 distance is 3.37 km, which rounds to the reported 3.4 km, so the displayed distance is consistent with a haversine edge cost on the stored coordinates.

TABLE V
GREAT-CIRCLE DISTANCES FROM STORED COORDINATES

Hospital pair	Distance (km)
HSP-0001 – HSP-0002	3.37
HSP-0001 – HSP-0003	3.37
HSP-0001 – HSP-0004	112.76
HSP-0002 – HSP-0003	0.00
HSP-0002 – HSP-0004	115.56
HSP-0003 – HSP-0004	115.56

E. Discussion

The demonstration shows the intended end-to-end flow: persisted records feed a matching result with an inspectable explanation and a transport plan with distance, time, and risk. It also exposes four points that matter for interpretation.

First, the risk and success values (51.4% and 48.6%) are complements and sum to 100%, as Section IV-D requires. A risk of 51.4% for a 3.4 km route is high for such a short transfer, which illustrates that the Bayesian parameters are uncalibrated prototype settings and not evidence-based probabilities.

Second, 3.4 km in 3 min implies an average speed of about 68 km/h. Because the distance is a straight-line distance, true road distance is at least as long, and urban speeds in Chennai are usually lower, so the time estimate is optimistic.

Third, the coordinates of HSP-0002 and HSP-0003 are identical (13.0827, 80.2707), giving a zero-km edge. These values are the example placeholders shown in the registration form, so they may be a data-entry artefact.

Fourth, in a complete graph whose edge cost is straight-line distance, the triangle inequality makes the direct edge optimal, so A* and UCS both return a two-node route. Non-trivial routing, in which A* expands fewer nodes than UCS, requires a sparse road-network graph.

F. Limitations

The evaluation is a single demonstration on a small seeded database, not a benchmark. The CSP score formula and maximum, the Bayesian variables and priors, and the details of the shipped UCS cross-check are not documented in the interface and must be confirmed from source code. Route times do not reflect live traffic or ambulance availability. A production system would need authentication, role-based access, audit logs, privacy safeguards, and formal clinical and regulatory review. Outputs are decision support only, and all clinical decisions must remain with authorised professionals.

VII. CONCLUSION AND FUTURE WORK

We presented IntelliTransplant, a decision-support prototype that unifies organ, recipient, and hospital records and combines a knowledge engine, a CSP engine with backtracking, A* and UCS route planning, a Bayesian risk estimate, and a coordination agent behind a React–FastAPI–SQL stack. In an end-to-end demonstration, the system returned a kidney match with an inspectable explanation and a 3.4 km transport plan whose distance we verified independently against the stored coordinates. The analysis also shows that the score range, the risk parameters, and the travel-time model need documentation and calibration before the outputs can be trusted.

Future work will document and unit-test the score formula, bounds, and tie-breaking; replace the complete graph with a road-network graph and compare A* and UCS on known test graphs; calibrate and validate the risk model with authorised data; enforce a single identifier format and explicit relations among organs, recipients, and hospitals; add authentication, role-based access, audit trails, and privacy-aware data handling; and add automated tests, richer error handling, and a usability study with transplant coordinators.

REFERENCES

[1] United Network for Organ Sharing, “How we match organs.” [Online]. Available: https://unos.org/transplant/how-we-match-organs/

[2] World Health Organization, “WHO guiding principles on human cell, tissue and organ transplantation,” 2024. [Online]. Available: https://www.who.int/health-topics/transplantation

[3] A. E. Roth, T. Sönmez, and M. U. Ünver, “Kidney exchange,” *Q. J. Econ.*, vol. 119, no. 2, pp. 457–488, 2004.

[4] R. Dechter, *Constraint Processing*. San Francisco, CA, USA: Morgan Kaufmann, 2003.

[5] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2020.

[6] A. K. Mackworth, “Consistency in networks of relations,” *Artif. Intell.*, vol. 8, no. 1, pp. 99–118, 1977.

[7] R. M. Haralick and G. L. Elliott, “Increasing tree search efficiency for constraint satisfaction problems,” *Artif. Intell.*, vol. 14, no. 3, pp. 263–313, 1980.

[8] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numer. Math.*, vol. 1, no. 1, pp. 269–271, 1959.

[9] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968.

[10] R. W. Sinnott, “Virtues of the haversine,” *Sky and Telescope*, vol. 68, no. 2, p. 159, 1984.

[11] D. L. Poole and A. K. Mackworth, *Artificial Intelligence: Foundations of Computational Agents*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2017. [Online]. Available: https://artint.info/2e/html/ArtInt2e.html

[12] N. J. Nilsson, *Artificial Intelligence: A New Synthesis*. San Francisco, CA, USA: Morgan Kaufmann, 1998.

[13] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. San Mateo, CA, USA: Morgan Kaufmann, 1988.

[14] T. Miller, “Explanation in artificial intelligence: Insights from the social sciences,” *Artif. Intell.*, vol. 267, pp. 1–38, 2019.

[15] FastAPI, “SQL (Relational) Databases.” [Online]. Available: https://fastapi.tiangolo.com/tutorial/sql-databases/

[16] SQLite, “SQLite documentation.” [Online]. Available: https://www.sqlite.org/docs.html

[17] Mozilla, “HTTP: Overview,” MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview